

Magertron™

Building an agent for Magertron

Credentials, discovery, tool calls, and delegation — for whoever writes the agent
Agent Developer Guide

DEVELOPER GUIDE · 2026-09-08

CONFIDENTIAL — FOR EVALUATION AND OPERATOR USE

Index

Select any entry to jump straight to it. The same entries appear as bookmarks in your PDF viewer's sidebar.

What you are building	3
1 · What you need from your platform team	3
2 · Five minutes: prove it works	3
3 · Calling a tool	4
4 · Acting for a person	5
5 · The loop	6
Cache the credential, respect the expiry	6
Fetch the catalogue as the person, not the intersection	6
Never degrade to one identity	6
Retry once on a stale 403 — and only once	6
6 · A working client	7
7 · When it goes wrong	7
8 · What the platform does that you do not	8

Building an agent for Magertron

This is for whoever writes the agent. It assumes your platform team has already registered you — if the words "exchange audience" mean nothing yet, that is their chapter, not yours; ask them for §1's four values and start here.

Everything below has been run against a live platform. Where an error message appears, it is the exact text you will see.

What you are building

Your agent calls tools through a gateway that decides, per call, whether the call may proceed. You do not implement any of that. What you implement is:

1. getting a credential,
2. discovering what you may call,
3. making calls with that credential,
4. handling the four ways it can go wrong.

The rest of this guide is those four things in order.

1 · What you need from your platform team

Value	Example	What it is
Gateway URL	<code>https://mcp.acme.com</code>	where you send everything
Client ID	<code>mXaCNuzq...</code>	your agent's identity in the directory
Client secret	(64 chars)	proves it
Exchange audience	<code>https://sts.acme.com</code>	the API identifier your tokens must carry

⚠ **The audience is not a URL you call.** It is a label the directory stamps into tokens. Nothing is hosted there.

Ask them to confirm one more thing: that the API registration allows **both** client access and user-delegated access. If it allows only the first, everything in §4 will fail with a message about resource servers, and it will not be obvious why.

2 · Five minutes: prove it works

Before writing any code, make one call by hand.

```
GW=https://mcp.acme.com
CLIENT_ID=...           # yours
CLIENT_SECRET=...       # yours
AUDIENCE=https://sts.acme.com
```

```
IDP=https://acme.eu.auth0.com/oauth/token

# 1 · your agent's token, from your directory
AGENT=$(curl -s -X POST "$IDP" -H 'Content-Type: application/json' \
  -d '{"grant_type":"client_credentials",
      "client_id":"$CLIENT_ID",
      "client_secret":"$CLIENT_SECRET",
      "audience":"$AUDIENCE"}' \
  | python3 -c 'import sys,json; print(json.load(sys.stdin)["access_token"])')
echo "agent token: ${#AGENT} chars"

# 2 · trade it for a Magertron credential
TOKEN=$(curl -s -X POST "$GW/api/v1/token/exchange" \
  -H 'Content-Type: application/json' \
  -d '{"grant_type":"urn:ietf:params:oauth:grant-type:token-exchange",
      "subject_token":"$AGENT",
      "subject_token_type":"urn:ietf:params:oauth:token-type:jwt"}' \
  | python3 -c 'import sys,json; d=json.load(sys.stdin); print(d.get("access_token") or d)')
echo "magertron token: ${#TOKEN} chars"

# 3 · what may you call?
curl -s "$GW/api/v1/portal/catalog" -H "Authorization: Bearer $TOKEN" | head -c 400
```

Three lengths and a catalogue means you are connected. If step 2 printed an error object instead of a token, jump to §7 — the message names the cause.

⚠ **Check the lengths.** An empty variable produces `Authorization: Bearer` and a 401 that reads like a permissions problem. This wasted an hour of the author's day; it will waste one of yours.

3 · Calling a tool

A tool call is JSON-RPC over HTTP. Two requests: open a session, then call.

```
NS=mcp-prod
SERVER=mags-dev-time

SID=$(curl -s -D - -o /dev/null -X POST "$GW/mcp/$NS/$SERVER/" \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -H 'Accept: application/json, text/event-stream' \
  -d '{"jsonrpc":"2.0","id":1,"method":"initialize","params":{"
    "protocolVersion":"2025-03-26","capabilities":{},
    "clientInfo":{"name":"my-agent","version":"1"}}}' \
  | grep -i '^mcp-session-id:' | cut -d' ' -f2 | tr -d '\r')

curl -s -X POST "$GW/mcp/$NS/$SERVER/" \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -H 'Accept: application/json, text/event-stream' \
  -H "Mcp-Session-Id: $SID" \
  -d '{"jsonrpc":"2.0","id":2,"method":"tools/call","params":{"
    "name":"get_system_time","arguments":{"timezone":"UTC"}}}'
```

⚠ **Some servers are stateless** and issue no `Mcp-Session-Id`. That is not a failure. Distinguish it from a real one by the initialize response: HTTP 200 with a JSON-RPC `result` means stateless — carry on without the header. Anything else is a refusal, and its body says why. The reference client makes exactly this distinction:

```
if sid == "" {
    if resp.StatusCode == http.StatusOK && initHasResult(raw) {
        return statelessSession, nil // live, nothing to carry
    }
    return "", fmt.Errorf("no session id (HTTP %d): %s", resp.StatusCode, ...)
}
```

⚠ **Authenticate the handshake too.** A handshake that authenticated as one identity and a call that authenticated as two is a seam nobody can explain.

4 · Acting for a person

Everything so far was your agent acting as itself — a complete, governed, audited call. Most agents need nothing more.

If your agent acts *for someone*, the call carries two identities and succeeds only where **both** are permitted. It can never reach further than your agent could alone; the intersection only narrows.

Add the person's token as the subject and yours as the actor:

```
POST /api/v1/token/exchange
{
  "grant_type": "urn:ietf:params:oauth:grant-type:token-exchange",
  "subject_token": "<the PERSON's token>", ← who it is FOR
  "subject_token_type": "urn:ietf:params:oauth:token-type:jwt",
  "actor_token": "<YOUR token>", ← who is ACTING
  "actor_token_type": "urn:ietf:params:oauth:token-type:jwt"
}
```

⚠ **The RFC's names read backwards.** The *subject* is the human; the *actor* is the agent. Reversing them produces a call that looks delegated and is not.

⚠ **The person's token must also carry your exchange audience.** If you get it from your directory, ask for a token *for the API* — not an `id_token`. An `id_token`'s audience is the client application and the exchange refuses it:

```
token verification failed: token doesn't contain the required audience
```

The result is an ordinary bearer token, valid **five minutes**, carrying both identities signed into it. Use it exactly as in §3. You cannot change who you act for without exchanging again — which is the point.

An agent may not act for an administrator. That delegation is refused outright rather than reduced:

```
403 on-behalf-of principal 'ops-admin' holds a non-delegable role (system:platform-admin): an agent may not act for an administrator
```

5 · The loop

Four rules. The first three are easy; the fourth is the one that bites.

Cache the credential, respect the expiry

Five minutes, and re-exchanging per call is wasteful. Refresh on a margin:

```
// A 30-second margin: enough that a token cannot expire in flight between
// this check and the upstream reading it.
if t.cached != "" && time.Now().Add(30*time.Second).Before(t.expires) {
    return t.cached, nil
}
```

Fetch the catalogue as the person, not the intersection

If you are delegating, ask "what can this person do?" with their credential alone. Answering with the intersection hides tools your agent happens to lack, which is a confusing answer to "what can I do here" — and every call is gated anyway, so nothing becomes reachable by being listed.

Never degrade to one identity

If the delegation cannot be established, fail the call. Falling back to "just the agent" is a privilege change in the wrong direction and looks identical to the feature working.

Retry once on a stale 403 — and only once

⚠ **This is the least obvious property of the system.** A token carries the roles its principal held when it was minted. A permission granted an hour ago does not reach a token issued two hours ago, and there is no push notification — the gateway validates a signature rather than consulting a session table.

So an agent running for hours will keep being refused for a permission it now has, and the refusal looks like a platform fault.

```
if resp.StatusCode == 403 && mentionsRolesOrPolicy(body) && !alreadyRetried {
    tok, _ = provider.Refresh(ctx) // re-exchange: roles are read fresh
    alreadyRetried = true
    continue
}
```

Once. An agent that retries a genuine denial indefinitely turns one refusal into a loop against the gateway.

⚠ It is easy to get this wrong while appearing correct. The reference daemon re-fetches its tool catalogue every turn, deliberately, with a comment saying entitlements can change. It still went stale — the *fetch* was fresh and the *identity* was not.

6 · A working client

The reference implementation is `services/mcp-chat-daemon/magertron.go` . Its shape, if you are writing your own:

Method	Does
<code>Login(user, pass)</code>	a person's session, when your agent signs someone in
<code>Catalog(userJWT)</code>	what may be called — the person's token alone
<code>OpenSession(..., ns, server)</code>	initialize; returns a session id or the stateless sentinel
<code>CallTool(..., sessionID, tool, args)</code>	<code>tools/call</code> , with the session id when there is one
<code>SAToken(ctx)</code>	the credential: cached, refreshed, exchange or static

One detail worth copying: the principal headers are set by a **single helper**, never inline per call site, because the identities must agree on every request.

```
func setPrincipalHeaders(req *http.Request, saJWT, userJWT, turnID string) {
    if turnID != "" && sendTurnID {
        req.Header.Set("X-Magertron-Turn-Id", turnID)
    }
    if saJWT != "" {
        req.Header.Set("Authorization", "Bearer "+saJWT)
        req.Header.Set("X-Magertron-On-Behalf-Of", "Bearer "+userJWT)
        return
    }
    req.Header.Set("Authorization", "Bearer "+userJWT)
}
```

⚠ That header form is the **in-cluster** shape, for a daemon that signed the person in itself and holds both tokens. If your agent runs anywhere else, use the exchange from §4: the binding is then established by the platform and signed, rather than asserted by you on each call, and you never hold anyone's session.

`X-Magertron-Turn-Id` is optional. Send a stable id per conversational turn and the platform groups a model call and the tool calls it produced into one unit — which is what makes a chargeback line read as an answer rather than as five unrelated calls.

7 · When it goes wrong

Each of these points somewhere other than its cause. That is why they are here.

token doesn't contain the required audience You sent an `id_token`. Ask your directory for an access token for the exchange audience. §4.

Client "..." is not authorized to access resource server "..." The API registration allows one access type and you need both. Your platform team's fix, not yours — send them §1.

No service account is provisioned for subject '...@clients' Your token verified; nobody recognises the identifier. Your `externalId` in Magertron must equal the `sub` your directory issues — Auth0 uses `<client_id>@clients`, Entra the service principal's object ID.

No user is provisioned for subject 'auth0|...' Same, for the person you are acting for. They must have signed in through the directory at least once, or been provisioned by SCIM.

The service account for '...' has been revoked. Your credential was rotated out. Re-exchange; if it persists, your identity was deprovisioned.

No permission for MCPServer/ViewLogs in 'mcp-prod'. Principal '...' has roles [...] Identity is fine, entitlements are not. The message names what was required and what you hold. A freshly provisioned identity arrives with **no roles at all** — that is expected, and someone grants them.

A 401 with an empty-looking request Check your token variable is non-empty before blaming the platform.

Everything works, but the audit shows one identity The delegation was not carried. Either you used a plain agent token where an exchanged one was intended, or a header was empty. Nothing degrades at the point of the check — but an agent that forgets to delegate makes a perfectly valid direct call instead, and no error says so.

8 · What the platform does that you do not

Worth knowing, so you do not build it yourself:

- **Credential injection.** You never see the vendor's API key. Your agent presents its own identity; the gateway attaches the credential upstream.
- **Tool approval.** If a vendor changes a tool's schema, calls stop until a human reviews the difference — regardless of your permissions.
- **Metering and chargeback.** Every call is attributed to the identities on it. Delegated calls bill the person, not the agent acting for them.
- **Budget.** A call that would exceed the contract's ceiling is refused with a 429, not billed and apologised for later.

None of this requires anything from your code beyond presenting a valid credential and handling a refusal gracefully.